Demo Client · DEMO-0012
11 November 2026 · v1.0

Field	Value
Document	Methodology of the data platform
Template version	data_engineering 1.0.0 / proquant-core 0.1.0
Engagement	DEMO-0012 · Demo Client
Period	2026-09-28 to 2026-09-30
Prepared by	ProQuant Analytics · info@proquantanalytics.com
Run ID(s)	de_20261111T142312Z_449cf0
Status	Final · 11 November 2026

1. Purpose

To describe the design of the platform and every rule it applies, so that another engineer can operate it, change it, or rebuild it.

2. Sources and how they are reached

Each source is declared once, with its type: a drop folder (files written by the source system or by a person), a read-only database query, an API (paged, with its authentication taken from the environment), a shared Google Sheet, cloud storage or SFTP. Every value that grants access is written as `${NAME}` and read from the environment or a `.env` file on the machine that runs the platform.

3. Load modes

- **New files:** each file is identified by the SHA-256 of its content. A file already loaded is not read again; the same content under another name is recorded as skipped. With `replace_by` (for example site and day), a file replaces what an earlier file loaded for the same partition, which is how corrected re-exports are absorbed.
- **Snapshot:** the source is a full export. It is loaded when its content changes and replaces the previous version of that source.
- **Incremental:** rows after the last watermark of a named column are read and upserted on the declared key.

4. Contracts

Before anything is written, each batch is checked against its contract: required columns present, at least the minimum number of rows, the key unique within the batch, and no more than the agreed share of rows with an unreadable required value. A failure sends the whole batch to quarantine with a written reason; nothing of it is loaded. Columns that appear or disappear are recorded as format changes; new columns are kept in the raw layer.

5. Layers

1. **Raw:** every batch as received, every value kept as text, with its source, batch, file name, SHA-256 and load time. Never edited, never deleted.

2. **Staging:** the standard tables of `core/intake` (transactions, items, places, expenses, staff, marketing, customers, targets, inventory). Each field has a fixed type; date parts (day, month, hour) are computed before loading so that every database groups by plain columns.
3. **Published:** dimensions (`dim_`), facts (`fact_`) and marts (`mart_`) built by the SQL files in `models/` .

6. Models

Each SQL file is one table: a plain SELECT, rebuilt from staging on every run. A header names the tables it needs, its meaning and the meaning of its columns, and its tests. Lines between `-- if TABLE` and `-- end` are used only when that table exists, so the same models serve a business that has sent only sales and one that has sent everything. The SQL avoids features specific to one database.

7. Tests on the published tables

After every build: unique keys, required values, references from facts to dimensions, and the sum of sales from staging to the line table, to the daily table, to the monthly table, which must agree to the cent. Data older than the agreed number of days, and sites behind the latest day, raise a warning.

8. Publishing

Every published table is written to `serving/` as CSV and Parquet with a manifest, for Power BI, Looker Studio or Excel, and stays in the warehouse for direct connections.

9. Runs

A run loads every source in the declared order, builds the calendar and the models, tests them, publishes, and records itself: start, status, batches loaded, skipped and set aside, rows, tables built, tests failed and warned. A run with nothing new must load nothing and change nothing. Exit codes: 0 ok, 2 warnings, 1 failed.

10. Portability

The warehouse is DuckDB in one file by default, or any database reached through SQLAlchemy (PostgreSQL, SQL Server, cloud warehouses with their driver). The engagement builds the same models a second time on SQLite from the same staging tables and tests them. Create-table scripts for PostgreSQL and BigQuery are generated from the published tables.

11. Operations

The platform package carries the engine, the configuration, the models, a self-test on made-up data, schedules (Windows Task Scheduler, launchd, cron, systemd, GitHub Actions) and the create-table scripts. The engagement unzips it into an empty folder and runs the self-test with no access to the build environment.

12. Readiness checks

Fifteen checks compare what the platform did with thresholds agreed at kickoff (`tests:`): how sources are collected, the last night's status and what it set aside, table tests, agreement of totals, freshness, format changes, a rerun with nothing new, portability, the package, the schedules, the scripts, the data dictionary, secrets in the configuration, and the time a night takes. Any FAIL gives "Not ready", any WARN "Ready, with actions".

13. Reproducibility

The run folder holds the configuration hash, the fingerprints of the inputs, the code version, every table as CSV and the log.

14. Limitations

The limits of this work are set out in the report. No deviations from the standard methodology.